Brainstormed Ideas to Enhance the System

Based on recent advancements in hybrid RAG and knowledge graph technologies (particularly from 2024-2025), here are several potential ideas to add value to your document query system. These build on the core strengths of precision from the knowledge graph (KG) and contextual depth from RAG, while addressing emerging trends like improved reasoning, multi-modality, and adaptability. I've organized them into categories for clarity, with explanations of how they could integrate and the value they might provide.

1. Incorporate Multi-Modal Data Handling

- **Idea**: Extend the system to support non-text documents by integrating multi-modal retrieval. For example, use embeddings for images, videos, or audio within documents (e.g., extracting visual entities via tools like CLIP models or OCR for scanned PDFs). The KG could link these to text entities (e.g., an image of a meeting agenda tied to a meeting node), while RAG pulls contextual descriptions.
- **Value-Add**: This would make the system more versatile for real-world document sets that include multimedia, improving comprehensiveness in queries like "summarize visuals from the project report." Recent hybrid RAG developments emphasize this for domains like legal or medical docs, where diagrams or scans are common. qed42.com
- **Implementation Tip**: Add a pre-processing step to generate multi-modal embeddings and store them in the vector store, with relationships in the KG.

2. Implement Agentic Workflows for Dynamic Query Handling

- **Idea**: Introduce AI agents (e.g., powered by LLMs) to orchestrate queries. An agent

could analyze the user's input, decide whether to prioritize KG for structured queries or RAG for semantic ones, refine ambiguous queries (e.g., via rephrasing), or chain multiple retrievals (e.g., first KG for entities, then RAG for context).

- **Value-Add**: Enhances user experience by handling complex, multi-step questions more intelligently, reducing errors and improving response relevance. This "agentic RAG" trend is gaining traction for its ability to adapt in real-time, making the system feel more interactive and robust for evolving document queries. ai.gopubby.com
- **Implementation Tip**: Use frameworks like LangChain to build agents that interact with your KG (e.g., via Cypher queries) and vector store, with fallback mechanisms for low-confidence results.

3. Add Hierarchical and Summarization Features (e.g., RAPTOR or Query-Focused Summarization)

- **Idea**: For long or complex documents, implement hierarchical processing like RAPTOR, which creates layered summaries (high-level overviews down to detailed chunks). Integrate this with the KG by extracting entities at each level and linking them. Alternatively, use query-focused summarization to generate KG subgraphs tailored to the query.
- **Value-Add**: Improves efficiency and accuracy for large datasets by avoiding information overload, while enabling better handling of "big picture" queries (e.g., "key themes across all meeting agendas"). Benchmarks show this boosts comprehensiveness by 70-80% in GraphRAG setups. ibm.com
- **Implementation Tip**: During ingestion, recursively summarize document sections and store summaries as nodes in the KG, with edges to original chunks in the vector store.

4. Enable Adaptive and Fusion-Based Retrieval

- **Idea**: Make retrieval adaptive by dynamically weighting KG vs. RAG based on query type

(e.g., more KG for factual/entity-based, more RAG for interpretive). Use fusion techniques like Reciprocal Rank Fusion (RRF) to combine results from both.

- **Value-Add**: Increases overall accuracy and recall, especially in mixed-query scenarios, with reported improvements of 8-9% in factual correctness from hybrid GraphRAG. arxiv.org This would make the system more resilient to diverse user needs without manual tuning.
- **Implementation Tip**: Train a simple classifier (or use LLM prompting) to score query complexity and adjust weights, leveraging existing indexing for speed.

5. Enhance Explainability and Traceability

- **Idea**: Add features to trace responses back to sources, such as visualizing KG paths (e.g., "This agenda links to Meeting X via entity Y") or providing confidence scores based on retrieval matches.
- **Value-Add**: Builds user trust, especially in professional settings like compliance or research, where explaining "why" a document was retrieved is crucial. KGs naturally support this through interpretable structures, aligning with trends in transparent AI. qed42.com
- **Implementation Tip**: Use graph visualization libraries (e.g., integrated with your database) to output paths alongside responses.

6. Incorporate Real-Time Updates and Feedback Loops

- **Idea**: Add streaming ingestion for new documents, automatically updating the KG and vector store via incremental NER and embedding generation. Include a user feedback mechanism (e.g., thumbs up/down) to refine relationships or retrain embeddings.
- **Value-Add**: Keeps the system current in dynamic environments (e.g., ongoing projects with frequent doc additions), improving long-term accuracy and personalization without full re-indexing.
- **Implementation Tip**: Use event-driven architectures (e.g., with Kafka) for updates, and store feedback as new edges in the KG for iterative improvement.

7. Explore Advanced Enrichment Techniques

- **Idea**: Use LLMs for graph enrichment, like inferring missing relationships (e.g., via prompt-based reasoning) or generating hypothetical documents (HyDE) to bridge gaps

prompt-based reasoning) or generating hypothetical documents (HyDE) to bridge gaps in sparse data.

- **Value-Add**: Reduces hallucinations and fills knowledge gaps, making queries more robust in incomplete datasets. This is a key 2025 trend for knowledge-intensive tasks.
- **Implementation Tip**: Run periodic enrichment jobs, validating new inferences before committing to the KG.

These ideas draw from cutting-edge developments and could be prioritized based on your specific use case (e.g., start with multi-modal if your docs are visual-heavy). They aim to make the system more accurate, user-friendly, and future-proof. If any resonate, we can dive deeper into implementation details!